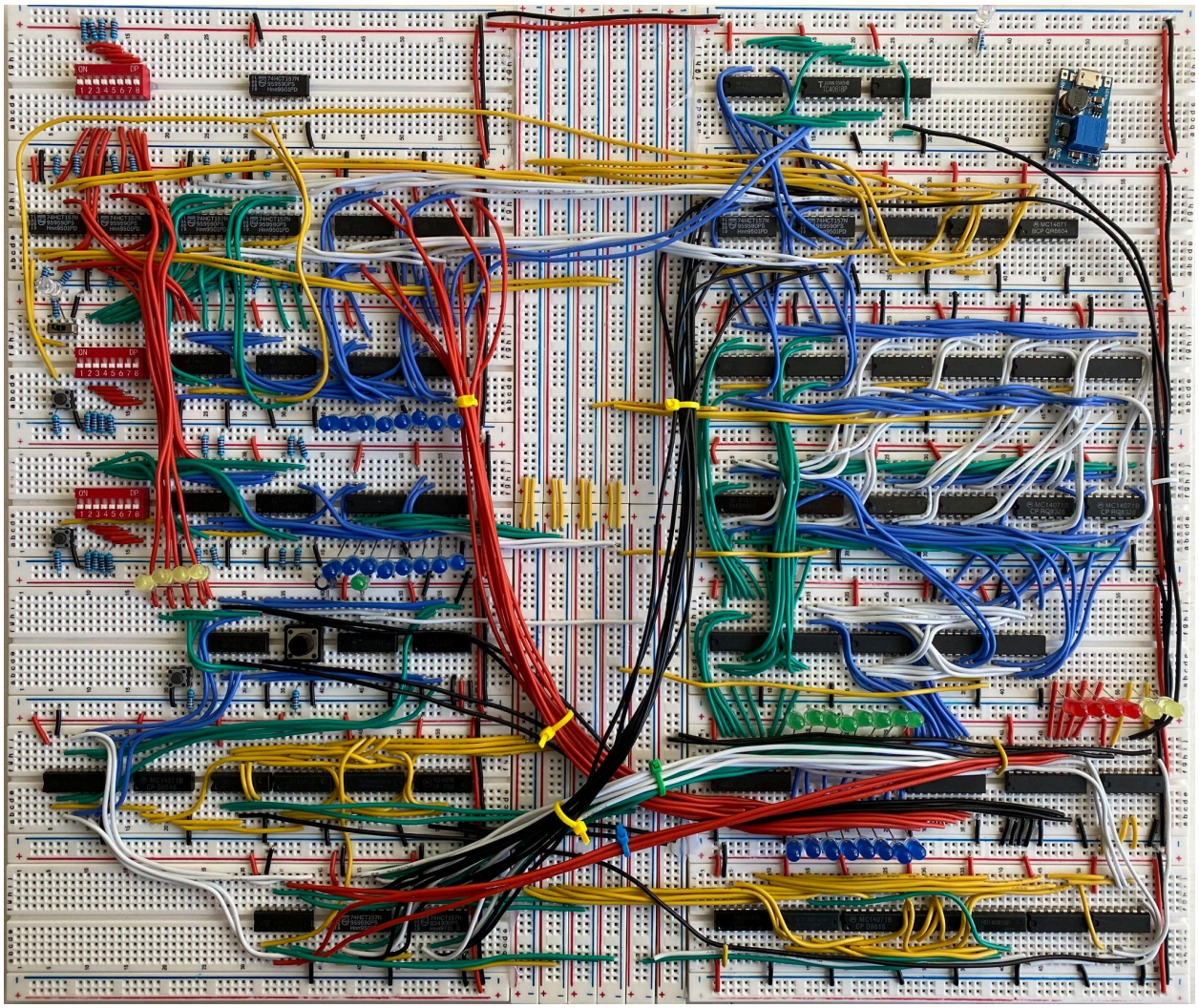
8 Bit Microprocessor Instruction Manual



Nicholas Grummon, January 2022

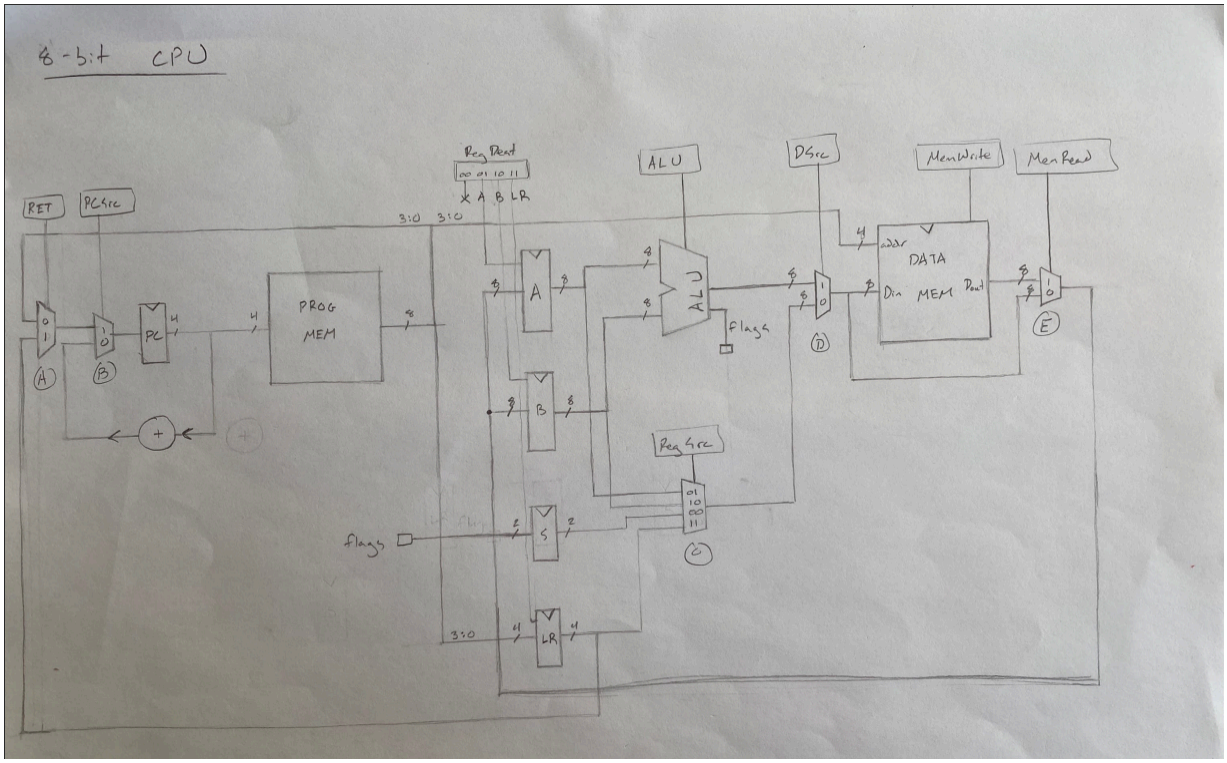
CONTENTS

Processor Specifications and Parts List

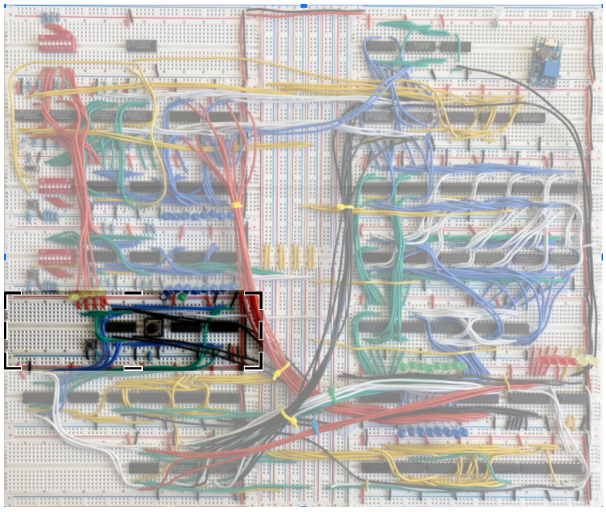
Dimensions	13in x 15.5in x 1in	
Power	5V, 1A supplied by USBC	
I/O	45 LEDs outputs 25 switch inputs 4 button inputs	
Memory	16 bytes data RAM 16 bytes program RAM 22 bit register file	
Timing	User pulsed clock (schmitt triggered) rising edge trigger for registers falling edge trigger for program counter	
Components	4	SN74189N 16 x 4 bit RAM
	3	74F283 full adder
	3	74HC574 octal type D flip flop (register)
	2	74HC175N quad type D flip flop (register)
	21	74HC157 quad two-to-one multiplexer
	1	74LS139 two-to-four decoder
	10	CD4081 quad two-input AND gate
	5	CD4071 quad two-input OR gate
	1	CD4070 quad two-input XOR gate
	10	74HC04N hex inverter
	3	74LS14N schmitt triggered hex inverter
	45	LED, assorted colors
	3	two prong mini push button
	1	four prong push button
	1	three prong switch
	3	8 bit dip switch block
	35	220 Ω resistor
	2	470 Ω resistor
	1	16 V 22 μF capacitor
	1	16 V 220 μF capacitor
	~100 ft	22 gauge wire, assorted colors

Processor Architecture

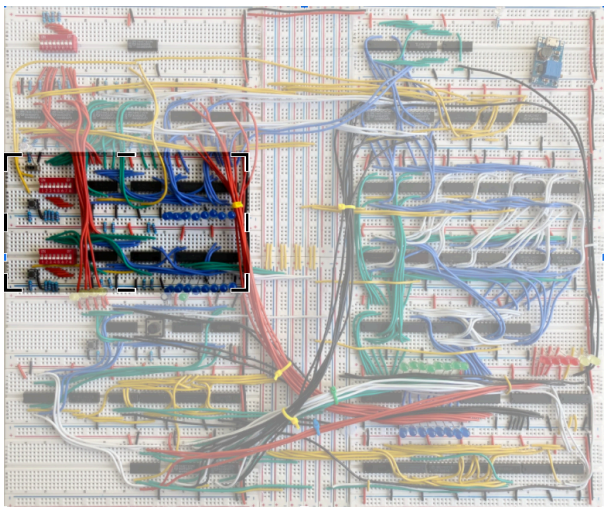
This CPU is a single-cycle processor based on ARM architecture. It has five core parts: a program counter, a program memory, a register file, an ALU, and a data memory. The program counter holds the address of the current instruction. It can be updated either by incrementing or by special branching instructions. The program memory holds all of the instructions used in a program. The address held by the program counter corresponds to an instruction in program memory. Register A and register B hold eight bit values that a program is currently using. The link register holds a four bit return address for the program counter, and the status register holds a two bit condition which is used to enable conditional branching. The ALU (arithmetic logic unit) performs all calculations (ADD, SUB, AND, OR, and NOT). It always takes register A and register B as inputs and always stores its result in register A. The final component, the data memory, holds all numerical values used by a program. The address space of an instruction corresponds to an address in data memory.



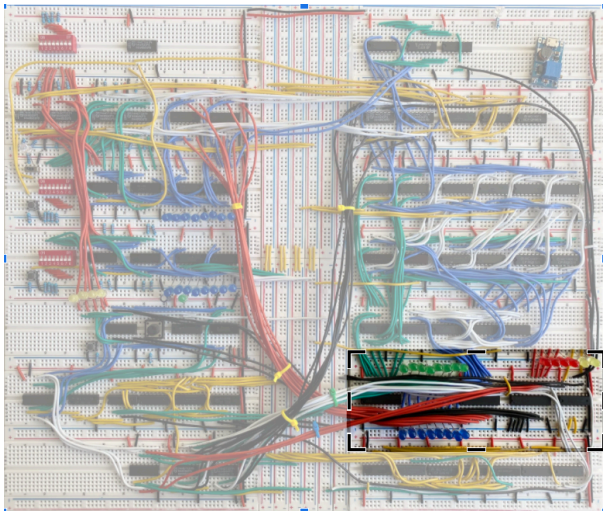
Clock and Program Counter



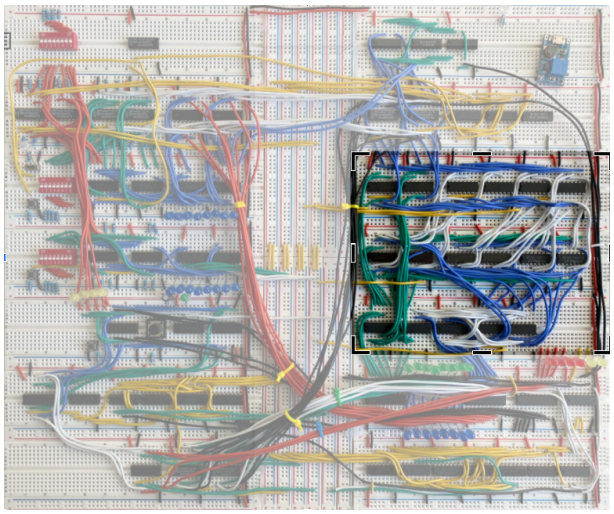
Memory (Program bottom, Data top)



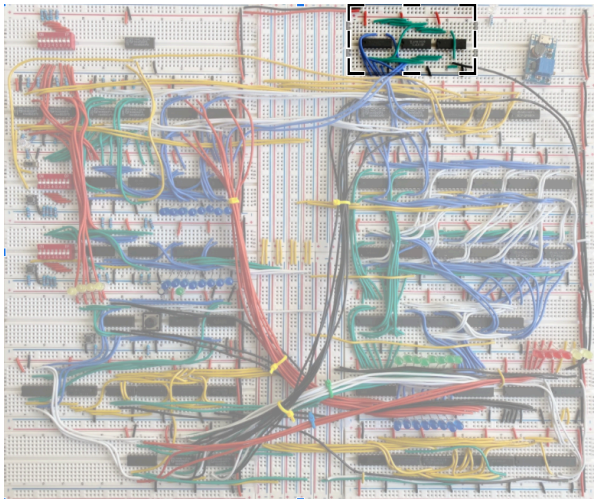
Register File



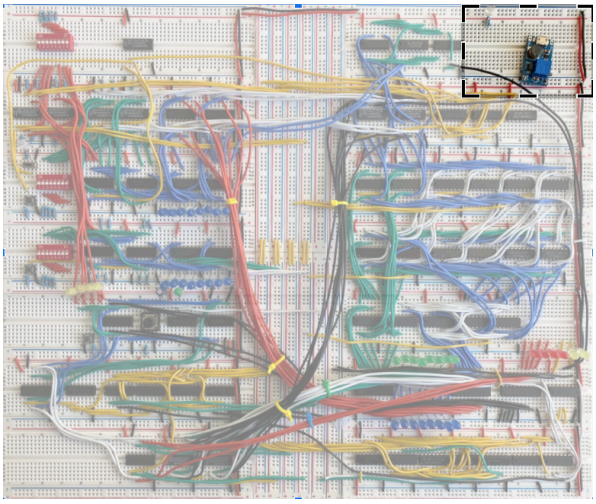
ALU



Flags



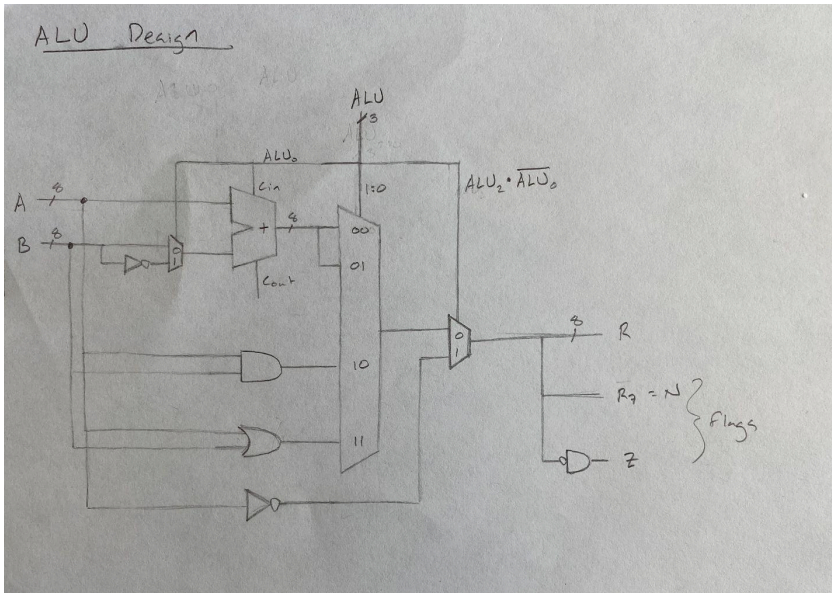
Power

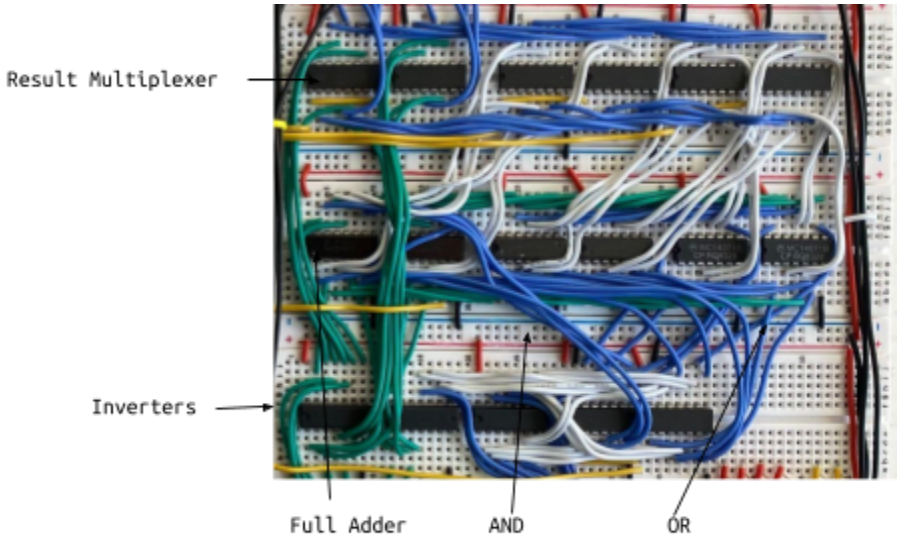


ALU Architecture

The processor features a five function ALU. These functions are addition, subtraction, bitwise AND, bitwise OR, and bitwise NOT. The ALU result is always stored into register A.

The ALU also features two flags, one for negative results and one for zero results. These flags are calculated for every ALU operation but are only used for the special CMP comparison command. This special operation subtracts register B from register A and sets the flags without storing a result in register A. Flags are stored in the status register.





Instruction Set

Each instruction has a four bit op-code and a four bit memory address as follows.

```
0010 1101
 op  addr
```

Data Processing Instructions

Data processing instructions have op-code `0000` and use address space for specifying operation because they do not interact with memory. All data processing instructions are executed in the ALU and store their result into register A.

Assembly	Instruction	Description
`ADD`	`0000 0000`	add regA + regB
`SUB`	`0000 0001`	subtract regA - regB
`AND`	`0000 0010`	bitwise AND(regA, regB)
`OR`	`0000 0011`	bitwise OR(regA, regB)
`NOT`	`0000 0100`	invert regA
`CMP`	`0000 1001`	subtract regA - regB and set flags, no result
`*LSR`	`0000 X1XX`	logical shift right by XXX bits

* LSR shifting function is supported by the machine language, but is not implemented in the physical processor

Memory Instructions

Memory instructions have op-code `0SRR` where S specifies whether to read or write from data memory and RR specifies which register to use. The address space of memory instructions specifies a data memory address.

Function	Instruction	Description
`STA`	`0001 XXXX`	store regA at data address XXXX
`STB`	`0010 XXXX`	store regB at data address XXXX
`LDA`	`0101 XXXX`	load data address XXXX into regA
`LDB`	`0110 XXXX`	load data address XXXX into regB
`*ADDS`	`0011 XXXX`	add regA + regB and store result at data address XXXX
`*SUBS`	`0111 XXXX`	subtract regA - regB and store result at data address XXXX

*ADDS and SUBS are supported by the machine language, but not implemented in the physical processor

Branching Instructions

Branching instructions have op-code `1CCL` where CC specifies a condition and L specifies whether or not to link a return address. Branching instructions only execute if the condition is true, meaning the condition bits match the status register. The address space of branching instructions specifies a program memory address.

Function	Instruction	Description
`B`	`1CC0 XXXX`	branch to program address XXXX
`BL`	`1CC1 XXXX`	branch to program address XXXX, store current address in LR
`RET`	`0100 ----`	load LR into PC, disregard address bits

Condition	`CC`	Code
unconditional		`11`
equivalent	`EQ`	`01`
less than	`LT`	`10`
greater than	`GT`	`00`

Controller Logic

The op-code of each instruction is used to generate control signals according to the following truth table. These control signals act as selector bits for the data flow multiplexers.

inst	op	addr	PCSrc	RegDest	RegSrc	ALU	DSrc	Mem Write	Mem Read	Ret	Stat
STA	0001		0	00	01	XXX	0	1	0	X	0
STB	0010		0	00	10	XXX	0	1	0	X	0
LDA	0101		0	01	XX	XXX	X	0	1	X	0
LDB	0110		0	10	XX	XXX	X	0	1	X	0
RET	0100		1	00	XX	XXX	X	0	X	1	0
B	1CC0		CC==S	00	XX	XXX	X	0	X	0	0
BL	1CC1		CC==S	11	XX	XXX	X	0	X	0	0
ADD	0000	0000	0	01	XX	addr	1	0	0	X	0
CMP	0000	1001	0	00	XX	addr	1	0	0	X	1

The following boolean equations are generated from the table:

PCSrc $F_7(F_6 \oplus S_1)'(F_5 \oplus S_0)' + \overline{F}_7 F_6 \overline{F}_5 \overline{F}_4 + F_7 F_6 F_5$

$RegDest_1$ $\overline{F}_7 F_6 F_5 \overline{F}_4 + F_7 F_4$

$RegDest_0$ $\overline{F}_7 \overline{F}_6 \overline{F}_5 \overline{F}_4 \overline{F}_3 + F_6 F_4$

$RegSrc_{1,0}$ F_5 , F_4

$ALU_{2:0}$ F_2 , F_1 , F_0

DSrc $\overline{F}_7 \overline{F}_6 \overline{F}_5 \overline{F}_4$

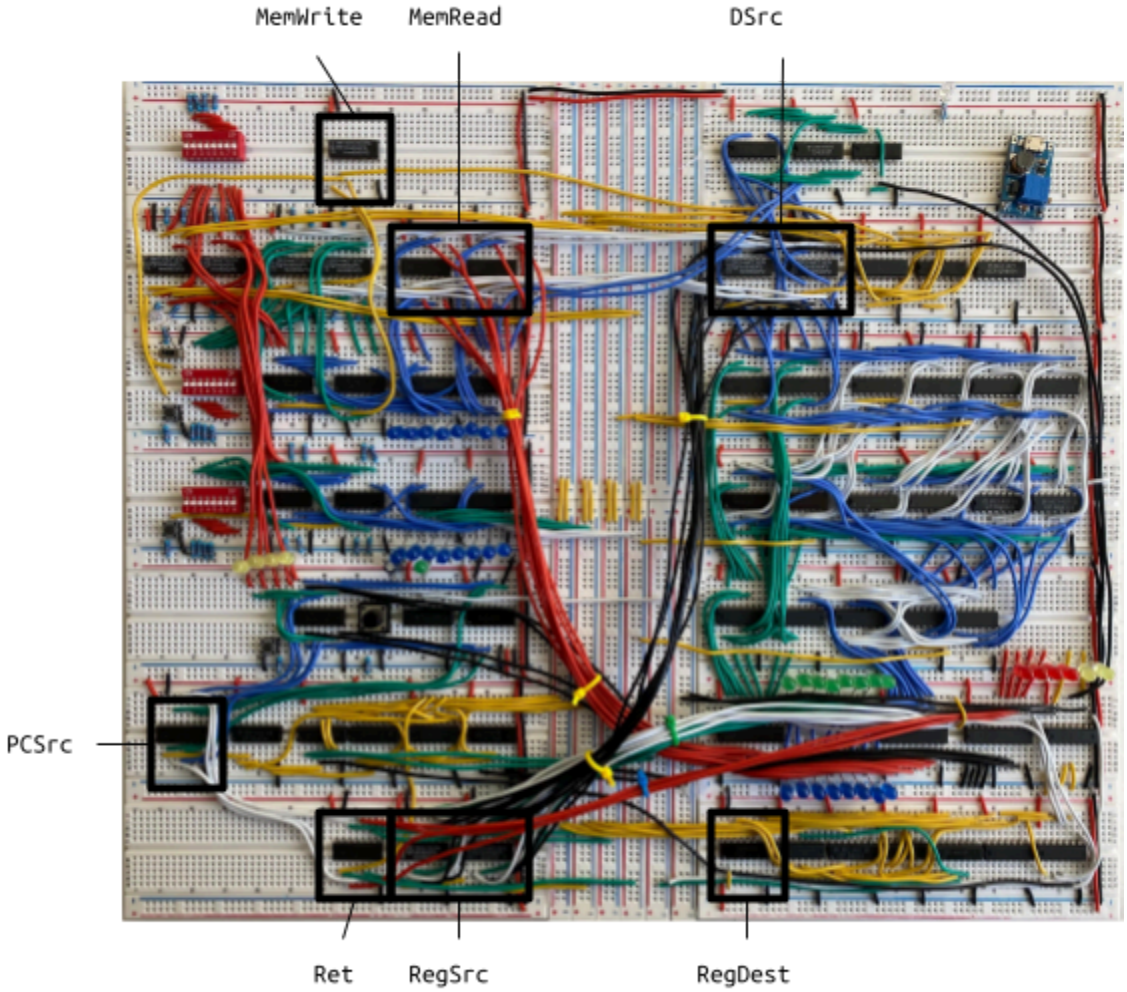
MemWrite $\overline{F}_7 \overline{F}_6 (F_5 + F_4)$

MemRead F_6

Ret $\overline{F}_7$

Stat $\overline{F}_7\overline{F}_6\overline{F}_5\overline{F}_4F_3$

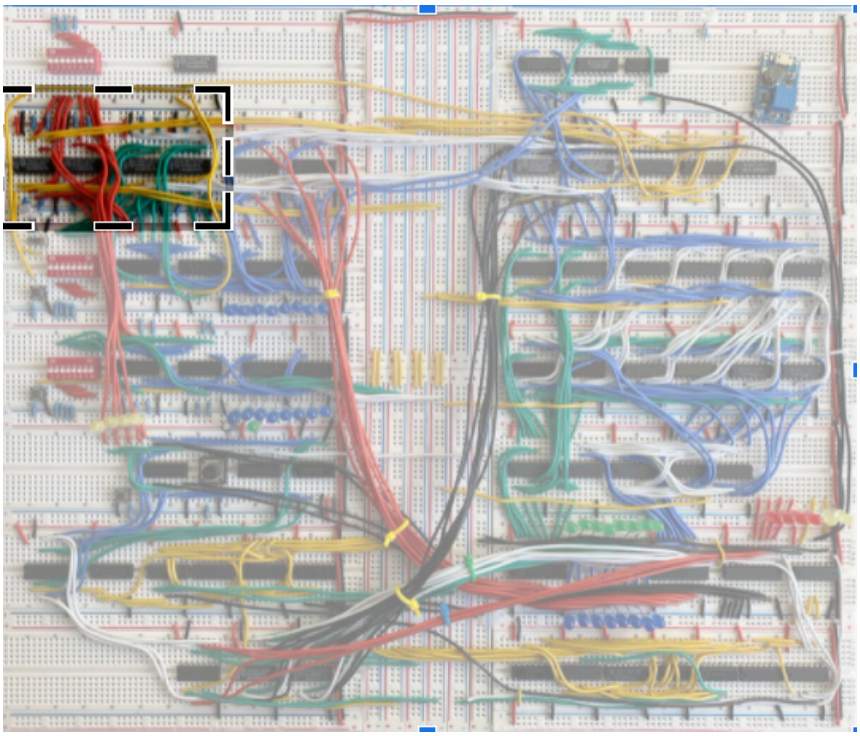
The data flow multiplexers are identified on the physical implementation below. The circuitry with yellow wiring next to these multiplexers is the associated control logic.



Mode Select and Manual Programming

The processor supports two modes - “programming mode” and “run mode.” In programming mode, the data and programming memories can be directly programmed with the three dip switch blocks and the associated “write” buttons. In run mode, the dip switches and write buttons are disabled and the memories can only be modified via store and load instructions. The mode is controlled by a slide switch on the left side of the board. A white status light next to this switch turns on when the processor is set to programming mode.

The mode select circuitry is highlighted below. It includes a series of multiplexers that choose between using the dip switches or the instructions for addressing and modifying memory.



The two lower dip switch blocks are used for entering bytes into the data and programming memories while the processor is in programming mode. Once the desired input is selected on the switches, the button to the left can be pressed to write to memory at the selected address and the switches can be changed again while the value is preserved in memory. The top block is used to change memory addresses while programming. The left four bits of this block address data memory, and the right four bits address program memory. More instructions for using the three dip switch blocks are included in the “User Guide” section of this manual.

Programming

The machine language used to encode instructions on this processor is translated directly from assembly language using the instruction tables on page six. While it is not necessary to include both assembly and machine languages in a sample code, it is very helpful for readability. Some specific syntax rules for assembly and machine language are given below.

Assembly Language

Instructions should be written first, followed by any conditions, followed by any immediates. Branch tokens should precede instructions on a line. Lines without a branch token should be indented. Immediates can either refer to a value or an address. If the immediate references an address, how the associated machine code is structured, it can simply be written using four bits or a hexadecimal character. If the immediate references a value which may be easier for readability, it needs to be identified with a pound sign. If assembly code is written by itself, then immediates should not reference addresses. All assembly codes must end with DONE: B DONE to avoid undefined end behavior.

Machine Language

Instructions should be preceded by the address where they are stored in memory. Data memory addresses should be put in brackets and program memory addresses should be put in parenthesis to ease the manual programming process. Data memory instructions should be written above the program. Program memory instructions must start at address 0000.

Example Programs

Ex 1. Add 3 + 5.

Assembly	Machine Code
`X = 3`	`[0000] 0000 0011`
`Y = 5`	`[0001] 0000 0101`
`LDA 0000`	`(0000) 0101 0000`
`LDB 0001`	`(0001) 0110 0001`
`ADD`	`(0010) 0000 0000`
`DONE: B DONE`	`(0011) 1110 0011`

Ex 2. Add 3 + 5. Output 1 if the result is greater than six. Otherwise output 0.

Assembly	Machine Code
`X = 3`	`[0000] 0000 0011`
`Y = 5`	`[0001] 0000 0101`
`N = 6`	`[0010] 0000 0110`
`P = 1`	`[0011] 0000 0001`
`Q = 0`	`[0100] 0000 0000`
`LDA #X`	`(0000) 0101 0000`
`LDB #Y`	`(0001) 0110 0001`
`ADD`	`(0010) 0000 0000`
`LDB #N`	`(0011) 0110 0010`
`CMP`	`(0100) 0000 0101`
`BGT IF`	`(0101) 1000 1000`
`ELSE: LDA #Q`	`(0110) 0101 0100`
`B DONE`	`(0111) 1110 1001`
`IF: LDA #P`	`(1000) 0101 0011`
`DONE: B DONE`	`(1001) 1110 1001`

Ex 3. Multiply 3 x 5.

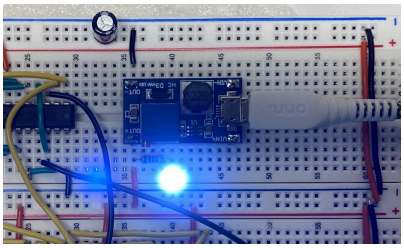
Assembly	Machine Code
X = 3	[0000] 0000 0011
Y = 5	[0001] 0000 0101
i = 1	[0010] 0000 0001
n = 1	[0011] 0000 0001
M = 0	[0100] 0000 0000
LOOP: LDA #n	(0000) 0101 0011
LDB #X	(0001) 0110 0000
CMP	(0010) 0000 1001
BGT END	(0011) 1000 1100
LDB #i	(0100) 0110 0010
ADD	(0101) 0000 0000
STA #n	(0110) 0001 0011
LDA #M	(0111) 0101 0100
LDB #Y	(1000) 0110 0001
ADD	(1001) 0000 0000
STA #M	(1010) 0001 0100
B LOOP	(1011) 1110 0000
END: LDA #M	(1100) 0101 0100
DONE: B DONE	(1101) 1110 1101

Ex 4. Sum the numbers less than 16.

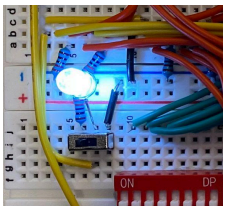
Assembly	Machine Code
i = 1	[0000] 0000 0001
n = 1	[0001] 0000 0001
N = 16	[0010] 0001 0000
S = 0	[0011] 0000 0000
LOOP: LDA #n	(0000) 0101 0001
LDB #N	(0001) 0110 0010
CMP	(0010) 0000 0101
BGT DONE	(0011) 1000 1100
LDB #S	(0100) 0110 0011
ADD	(0101) 0000 0000
STA #S	(0110) 0001 0011
LDA #n	(0111) 0101 0001
LDB #i	(1000) 0110 0000
ADD	(1001) 0000 0000
STA #n	(1010) 0001 0001
B LOOP	(1011) 1110 0000
DONE: B DONE	(1100) 1110 1100

User Guide

Plug in a USBC power source rated at least 1A to the module in the top right corner of the board. The white status LED next to this component should turn on to indicate that the board has power. Some additional LEDs may turn on if there is any residual data in the processor's memory.

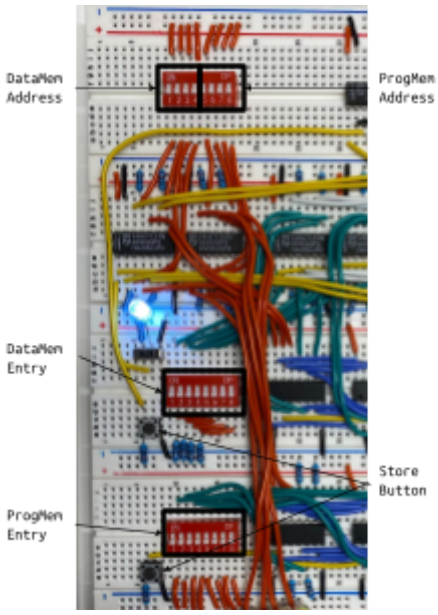


Locate the silver mode select switch on the left side of the board and toggle it to the left. The white status LED next to this component should turn on to indicate the processor is in programming mode.

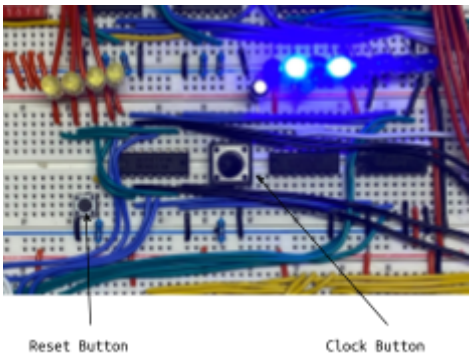


Use the three dip switch blocks to enter data into the processor's memory. The top block is for selecting an address, the middle block is for data memory, and the bottom block is for program memory. Push the button next to a block to write a value into memory. The eight blue lights to the right of the corresponding block should turn on to match the value entered.

After programming the processor, toggle the mode select switch to the right to enter run mode. Notice that the value now displayed by the blue program memory LEDs should correspond to the value stored at the address displayed on the yellow lights just below the programming block.

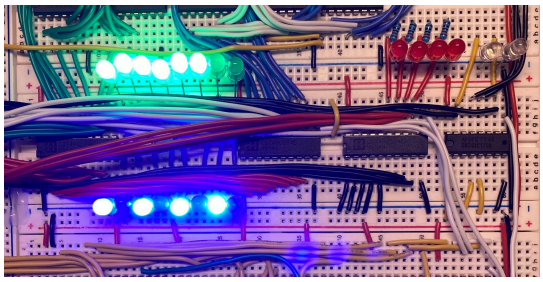


Push the button beneath the yellow lights to reset the program counter so that the processor starts on the first instruction. The yellow LEDs should all turn off and the first instruction should now be displayed by the memory LEDs along with the corresponding data value specified by the instruction's address space.



Locate the large clock button. Push this button once to execute the first instruction. A green light should flash just above the button to indicate that the clock pulsed. If the instruction modified a register file, the corresponding LEDs should update. As the clock button is released, the program counter LEDs should update, and a new instruction and associated data value should appear on the memory LEDs.

The register file is shown to the right. Register A is green, register B is blue, the link register is red (unlit), and the status register is white (unlit). Program output is displayed by the registers.



Continue pushing the clock button until the program terminates. Toggle the mode select switch to programming mode and either reprogram or unplug the processor. To turn off the processor, it is safe to simply unplug the USBC cable. A capacitor bridging the voltage lines protects against surges.

About the Design

The design of this processor is derived from the ARM architecture discussed in Vanderbilt's EECE 2123 Digital Systems course. The schematic on page 3 of this manual is the original layout for the processor. Details, such as the number of registers or available RAM, are loosely based on convenience but are ultimately arbitrary decisions that informed the rest of the design.

The processor features several items, such as a link register and a six function ALU, that are not strictly necessary for a computationally complete device, but the goal of this project was to implement a full-scale processor. Even though the potential of these features can't be fully appreciated with only eight bits, it was important to include them anyway for the sake of implementing everything a "real" computer can. That being said, neither an oscillating clock nor a very convenient I/O system were included because the project was more concerned with the digital systems actually driving a computer.

The instruction set for the processor was laid out after the basic architecture was finished. It needed to include at least a store and load command for each register as well as branch, branch and link, and return commands. The data processing instructions chosen to be included in the design followed those included in the four-function ALU discussed in class. An additional invert command was also included for register A, and room for a CMP comparison command was reserved in the eight bit instruction space, although no additional hardware in the ALU would be required to implement this.

Next, the machine code for each instruction was developed. Four bits had to be reserved for memory addressing since the processor has 16 words in each memory. That leaves four bits for op-codes to define instructions. The processor only has 13 instructions, so op-codes could have been arbitrarily assigned in numerical order, but to increase organization, instructions in the same family (data processing, memory, etc) were numbered close to each other in the instruction space. For example, all branching instructions have op-codes that start with 1. The machine code can be further organized by realizing that ALU instructions do not interact with the memory, so address space can actually be used for data-processing op-codes.

After these largely arbitrary decisions for the machine code framework, the specific op-codes were designed with simplicity of generating control signals in mind. For example, the ALU carry-in bit needs to be set for subtraction but cleared for addition. This is conveniently dictated directly by the LSB of the machine code.

Next, the required control signals were determined for each instruction. For example, the store command needs to set the MemWrite flag, etc. The full table of these flags is included in the Controller Logic section of this report. The boolean equations for each flag were derived, and all that was left was assembling the processor.